

Midterm Essay

Course: Massive Data Processing

1st Nguyen Triet Hung
Faculty of Information Technology
Ton Duc Thang University
522h0144@student.tdtu.edu.vn

3th Hoang Le Minh Nhat
Faculty of Information Technology
Ton Duc Thang University
523h0064@student.tdtu.edu.vn

2nd Truong Nguyen Bao
Faculty of Information Technology
Ton Duc Thang University
522h0166@student.tdtu.edu.vn

4th Le Nguyen Hai An
Faculty of Information Technology
Ton Duc Thang University
522h0027@student.tdtu.edu.vn

Abstract—Abstract This report presents the midterm assignment for the Massive Data Processing course, focusing on mining frequent patterns from the “baskets.csv” dataset using distributed computing frameworks. Three tasks are explored: (1) applying the “A-Priori” algorithm with Hadoop MapReduce to identify frequent customer pairs shopping on the same date, (2) implementing the “PCY algorithm” with “PySpark DataFrames” to detect frequent item pairs and generate association rules based on support and confidence thresholds, and (3) Using the FPGrowth algorithm, existing in PySpark, to discover frequent pairs and association rules based on a given support threshold s and a confidence threshold c .

Index Terms—Massive Data Processing

I. INTRODUCTION

This project, part of the Introduction to Big Data course at Ton Duc Thang University, involves implementing three tasks to explore Big Data techniques. Conducted by a group of four students over two weeks, the project uses Python in Visual Studio Code and Google Colab, with the report composed in Overleaf using the IEEE template. The tasks include: (1) Set up a Hadoop system in the fully-distributed mode, use Docker containers to execute the Word Count job to verify your installation and implement the A-Priori algorithm to identify frequent pairs in form of Hadoop MapReduce programs; (2) Use RDD (without using DataFrame and with using DataFrame) and apply PCY algorithms to complete the problem requirements; (3) Using the FPGrowth algorithm, existing in PySpark, to discover frequent pairs and association rules based on a given support threshold s and a confidence threshold c .

II. TASK 1: Hadoop & A-Priori Algorithm

First Pass: Identifying Frequent Customers

Understand and implement MapReduce program for Market Basket Analysis – find product pairs that are often bought together, based on input data which is a list of transactions (baskets.csv).

The Mapper function (CustomerMapper) reads each line of the transaction data and emits a key-value pair of (customer ID, 1) for each record. It first skips the header row to avoid processing column names and then splits each line by commas to extract the customer ID (Member number). For every occurrence, it emits the customer ID along with the count of 1.

The output of this first pass is a list of frequent customers, which serves as input for the second pass to detect pairs of frequent customers that shop together on the same date.

```
haian2004@haian2004-virtual-machine:~$ java -version
openjdk version "1.8.0_452"
OpenJDK Runtime Environment (build 1.8.0_452-8u452-ga-us1-0ubuntu1~20.04-b
99)
OpenJDK 64-Bit Server VM (build 25.452-b09, mixed mode)

haian2004@haian2004-virtual-machine:~$ jps
8816 DataNode
8645 NameNode
9194 ResourceManager
9338 NodeManager
9677 Jps
9021 SecondaryNameNode

haian2004@haian2004-virtual-machine:~$ hdfs dfs -cat /user/haian2004/output/part-r-00000
hadoop 1
hello 2
mapreduce 1

haian2004@haian2004-virtual-machine:~$ hdfs dfs -mkdir -p /user/hadoop/input
haian2004@haian2004-virtual-machine:~$ hdfs dfs -put ~/Downloads/baskets.csv /user/hadoop/input/
haian2004@haian2004-virtual-machine:~$ hdfs dfs -ls /user/hadoop/input
Found 1 items
-rw-r--r-- 1 haian2004 supergroup 1566578 2025-10-20 00:09 /user/hadoop/input/baskets.csv

haian2004@haian2004-virtual-machine:~$ hdfs dfs -cat /user/hadoop/output1/part-r-00000 | head
Instant food products 60
UHT-milk 323
abrasive cleaner 22
artif. sweetener 29
baby cosmetics 3
bags 4
baking powder 122
bathroom cleaner 17
beef 516
berries 327

haian2004@haian2004-virtual-machine:~$ hdfs dfs -cat /user/hadoop/output2/part-r-00000 | head -n 20
sausage,semi-finished bread 1
sausage,whole milk 1
sausage,yogurt 1
semi-finished bread,whole milk 1
semi-finished bread,yogurt 1
whole milk,yogurt 1
pastry,salty snack 1
pastry,whole milk 1
salty snack,whole milk 1
canned beer,misc. beverages 1
hygiene articles,sausage 1
pickled vegetables,soda 1
rolls/buns,sausage 1
rolls/buns,whole milk 1
sausage,whole milk 1
curd,frankfurter 1
soda,whole milk 1
beef,white bread 1
frankfurter,soda 1
frankfurter,whipped/sour cream 1
```

Fig. 1. A-Priori Execution on Hadoop

The Reducer function (CustomerReducer) is responsible for aggregating the number of occurrences of each customer and filtering out those who do not meet a predefined minimum support threshold.

Specifically, for each unique customer ID received from the mapper, the reducer receives a list of integers (typically a series of 1s) representing the number of times that customer appears in the dataset.

It then calculates the total count by summing these values. If the total count is greater than or equal to the minimum support (in this case, 3), the reducer emits the customer ID along with the count.

This step ensures that only frequent customers are retained for further analysis in the next MapReduce pass.

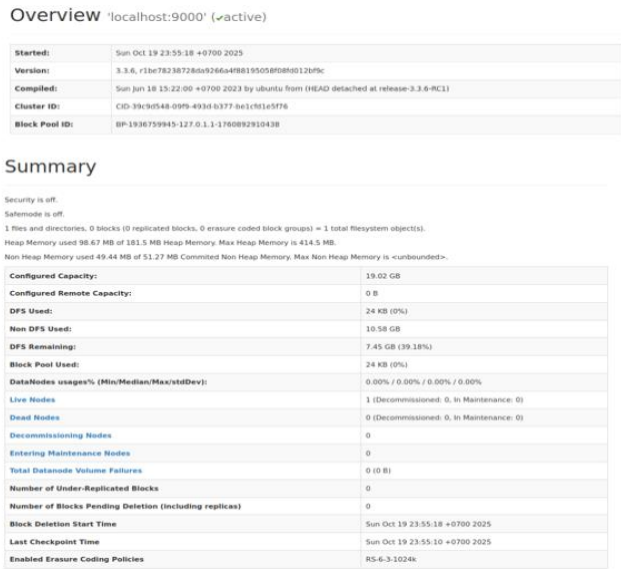


Fig. 2. Pseudocode of CustomerReducer Function

I.Second Pass: Identifying Frequent Customer Pairs

Finding frequent client couples that frequently shop together on the same day is a critical function of the Second-Pass in this MapReduce-based Apriori mining work.

The output from the First-Pass, which includes the list of clients who have purchased more than a certain threshold known as MIN-SUPPORT, is read by the program at the start. In the Mapper's setup() step, this list is loaded into memory as a Set.

The program then determines if the client is a member of the frequent customer set for each line of transaction data, which contains the customer ID and transaction date. If so, it groups clients according to the date they made their purchase by emitting a key-value pair of (date, customerID).

All consumers who made transactions on the same day are gathered by the software in the Reducer. From the list for that day, it then creates every potential customer pair and determines how frequently each pair has occurred throughout the entire set of data. Pairs are kept and recorded to the output if they reach or surpass the MIN-PAIR-SUPPORT criterion.

Customers that purchase together frequently, like friends or relatives, can be identified using this method. This information can then be used for marketing strategies, group promotions, or individualized customer support.

NameNode Journal Status

Current transaction ID: 1	
Journal Manager	State
FileJournalManager(root=/usr/local/hadoop/data/dfs/namenode)	EditLogOutputStream(/usr/local/hadoop/data/dfs/namenode/current/edits_inprogress_0000000000000000001)

NameNode Storage

Storage Directory	Type	State
/usr/local/hadoop/data/dfs/namenode	IMAGE_AND_EDITS	Active

DFS Storage Types

Storage Type	Configured Capacity	Capacity Used	Capacity Remaining	Block Pool Used	Nodes In Service
DISK	19.02 GB	24 KB (0%)	7.45 GB (39.18%)	24 KB	1

Hadoop, 2023.

Fig. 3. Pseudocode of Setup and Map Function
The goal is to focus on frequent customers only and emit (date, customerID) pairs to group by transaction date.

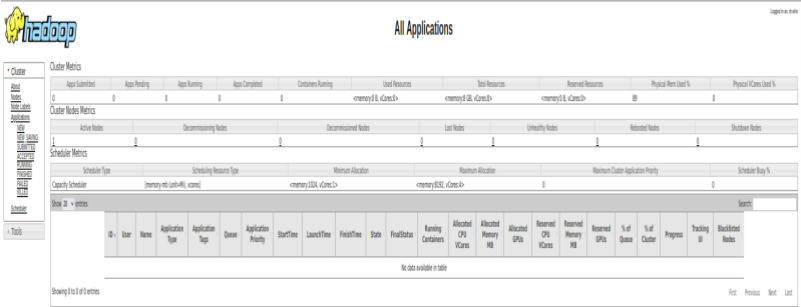


Fig. 4. Pseudocode of Reduce and Cleanup Function

III. TASK 2: PySpark & PCY Algorithm

In Task 2a and 2b, the group performs data analysis on the baskets.csv dataset using PySpark, applying two different approaches: RDDs and DataFrames. The main objective is to extract, summarize, and visualize shopping-related information from the transaction data. Specifically, the implemented functions identify the most frequently purchased items, calculate the number of shopping baskets for each customer, analyze monthly purchase trends, and track the purchasing activity of a specific customer over time. The results are displayed on the screen, saved into corresponding folders, and visualized using bar charts to provide a clear view of shopping trends.

A. Data Processing

In this task, the data preprocessing step focuses on preparing and structuring the transaction data in the baskets.csv file before performing analytical operations. The dataset contains essential information such as customer ID (Member_number), purchase date (Date), and purchased item (itemDescription).

The data was first loaded into PySpark using two different approaches — RDDs and DataFrames — to demonstrate the flexibility of PySpark in handling large-scale datasets. During processing, redundant columns were removed, and date-related fields such as year and month were extracted to facilitate time-based analysis.

Subsequently, the data was grouped and aggregated according to the analytical requirements of each function:

- Counting the total number of purchases for each item (f1).
 - Determining the number of baskets purchased per customer (f2).
 - Summarizing monthly purchase frequencies (f3).
 - Tracking the monthly purchase activity of a specific customer (f4).
- Finally, the processed results were displayed, saved into corresponding

folders, and visualized using bar charts to provide clear insights into purchasing patterns.

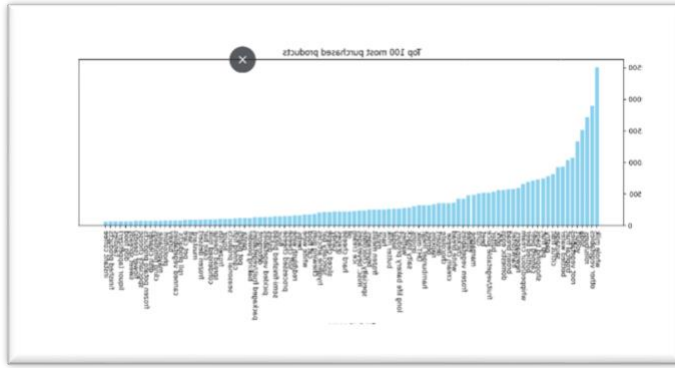


Fig. 6. Top 100 most purchased items

B. Function load file

```
Function load_file(sc, path)
  rdd ← sc.textFile(path)
  header ← rdd.first()
  data ← rdd.filter(each_line ≠ header)
  data ← data.map(split each_line by ",")
  Return data
End Function
```

Fig 7. Load file

C. Function draw bar chart

```
Function plot_chart(x, y, title, xlabel, ylabel, save_path, chart_type)
  Create directory for save_path if it does not exist
  Initialize a new figure with size (12, 6)

  If chart_type = "bar" Then
    Draw bar chart of x and y with color "skyblue"
  Else If chart_type = "line" Then
    Draw line chart of x and y with marker "o" and color "orange"
  End If

  Set chart title to title
  Set X-axis label to xlabel
  Set Y-axis label to ylabel
  Rotate X-axis labels by 90 degrees
  Adjust layout to fit labels
  Save figure to save_path
  Display the chart
End Function
```

Fig 8. Draw bar chart

D. PCY Algorithm

The "PCY (Park-Chen-Yu)" Algorithm is implemented in the "PCYAlgorithm" class, which takes four main parameters: the list of transactions, the bucket size, the support threshold, and the confidence threshold. This algorithm works in two main passes. In the first pass, the algorithm counts the frequency of individual items and hashes item pairs into buckets, whereas in the second pass, it filters frequent item pairs and generates association rules.

a) Setup

Upon initializing the "PCYAlgorithm" class, the subsequent elements are configured:

- "self.buckets": The count for each is included in this list. bucket, into which pairs of items will be hashed.
- "self.item-counts": A dictionary that keeps track of how frequently each item occurs.
- "self.frequent-items": A collection of things that are deemed "frequent" in the data because their frequency is higher than or equal to the support threshold.
- "self.freq-pairs": Depending on the outcomes of the next stages, this list contains frequent item pairs.
- "self.rule": A list of the association rules that were produced from the often occurring item pairings. Every

rule will be expressed as "A => B" or "B => A."

b) Steps in PCY Algorithm is:

- "First Pass: Hashing Item Pairs and Counting Frequencies": The algorithm iterates over each transaction in this step. For every transaction:
 - It initially updates "self.item-counts" by counting the frequency of each item.
 - The count for the appropriate bucket in "self.buckets" is then updated after iterating through every pair of items in the transaction and hashing them using the hash function.
- Filtering Frequent Items: Following the initial pass, the algorithm compares the items with the support threshold to filter them according to their frequency. If the frequency of the items is more than or equal to the support values are kept in "self.frequent-items" and are regarded as frequent.
- Finding Frequent Item Pairs: The method iterates through every transaction in the second pass to find frequent item pairs by determining whether both items in each pair are present in "self.frequent-items." A pair is deemed frequent if both items are frequent and the hash value of the pair falls into a bucket with a count higher than or equal to the support threshold. "self.freq-pairs" is where these frequently occurring pairs are kept.
- Generating Association Rules: Based on the frequent item pairs, the algorithm generates association rules. The rules are of the form A => B or B => A. For each frequent pair, the confidence of the rule is calculated as the ratio of the frequency of the pair to the frequency of item A (or B). If the confidence is greater than or equal to the specified threshold, the rule is stored in "self.rule".

E. Test PCY Algorithm

We utilize a small collection of test data (transactions) to test the "PCY algorithm," mimicking transaction data with a few transactions. Frequent item pairings and association rules are computed by the algorithm and printed for validation. Before using the algorithm on bigger datasets, we can make sure it performs as intended using this test data.

```
tropical fruit => whole milk, 0.13
0    pot plants => whole milk, 0.15
1    salty snack => rolls/buns, 0.11
2    canned beer => whole milk, 0.14
3    detergent => rolls/buns, 0.12
4    sweet spreads => tropical fruit, 0.22
['tropical fruit => whole milk, 0.13']
```

Fig. 7. Test results on small data

We apply the algorithm to actual transaction data taken from the "basket.csv file" after testing it on a small sample of data. Frequent item pairings and association rules are constructed and printed, and the transactions are sorted and processed similarly to the test data. This last phase shows how a huge dataset can be analyzed using the "PCY algorithm" to find significant correlations between items.

tropical fruit => whole milk, 0.13
 pot plants => whole milk, 0.15
 salty snack => rolls/buns, 0.11
 canned beer => whole milk, 0.14
 detergent => rolls/buns, 0.12
 sweet spreads => tropical fruit, 0.22
 meat => other vegetables, 0.13
 frozen meals => other vegetables, 0.13
 pip fruit => other vegetables, 0.11
 ham => other vegetables, 0.11
 soft cheese => whole milk, 0.12
 pot plants => other vegetables, 0.13
 sliced cheese => whole milk, 0.11
 ice cream => rolls/buns, 0.12
 berries => other vegetables, 0.13
 soda => other vegetables, 0.11
 processed cheese => root vegetables, 0.11
 other vegetables => whole milk, 0.13
 waffles => other vegetables, 0.11
 detergent => whole milk, 0.18
 grapes => whole milk, 0.14
 soft cheese => rolls/buns, 0.11
 pork => other vegetables, 0.11
 UHT-milk => other vegetables, 0.11
 sausage => soda, 0.10
 bottled water => whole milk, 0.13
 shopping bags => other vegetables, 0.11
 beverages => whole milk, 0.12
 detergent => yogurt, 0.12
 hamburger meat => whole milk, 0.15
 chewing gum => whole milk, 0.15
 bottled beer => whole milk, 0.16
 hard cheese => whole milk, 0.13
 yogurt => whole milk, 0.14
 chocolate => rolls/buns, 0.12
 citrus fruit => whole milk, 0.14
 white wine => whole milk, 0.11
 soft cheese => yogurt, 0.13
 frozen fish => whole milk, 0.17
 flour => whole milk, 0.17
 soft cheese => other vegetables, 0.13
 white bread => other vegetables, 0.11
 white bread => whole milk, 0.16
 curd => other vegetables, 0.11
 oil => whole milk, 0.14
 pork => whole milk, 0.15
 long life bakery product => whole milk, 0.13
 sausage => whole milk, 0.16
 hygiene articles => whole milk, 0.14
 pip fruit => rolls/buns, 0.11
 seasonal products => rolls/buns, 0.14
 specialty bar => whole milk, 0.13
 flour => tropical fruit, 0.11
 grapes => other vegetables, 0.13
 dessert => whole milk, 0.11
 specialty bar => other vegetables, 0.13
 chewing gum => yogurt, 0.12

Fig. 8. Result with support = 14 963 and confidence = 0.1

Member_number	date	items
1000	15/03/2015	[sausage, whole milk, semi-finished bread, yogurt]
1000	24/06/2014	[whole milk, pastry, salty snack]
1000	24/07/2015	[canned beer, misc. beverages]
1000	25/11/2015	[sausage, hygiene articles]
1000	27/05/2015	[soda, pickled vegetables]
1001	02/07/2014	[sausage, whole milk, rolls/buns]
1001	05/02/2015	[frankfurter, curd]
1001	12/12/2014	[whole milk, soda]
1001	14/04/2015	[beef, white bread]
1001	20/01/2015	[frankfurter, soda, whipped/sour cream]
1002	02/09/2014	[frozen vegetables, other vegetables]
1002	26/04/2014	[butter, whole milk]
1002	26/04/2015	[tropical fruit, sugar]
1002	30/08/2015	[butter milk, specialty chocolate]
1003	02/10/2015	[sausage, rolls/buns]
1003	15/10/2014	[root vegetables, detergent]
1003	19/01/2014	[frozen meals, dental care]
1003	27/02/2014	[rolls/buns, rolls/buns]
1004	02/04/2014	[pip fruit, whole milk, tropical fruit]
1004	05/01/2014	[dish cleaner, cling film/bags]

only showing top 20 rows

Number of frequent pairs: 12
 Number of association rules: 131

Fig. 9. Result with support = 14 963 and confidence = 0.1

V. TASK 3: FBGrowth Algorithm

In this section, requires applying the FPGrowth algorithm, existing in PySpark library, to discover frequent pairs and association rules based on a given support threshold s and a confidence threshold c and Implement a distributed program to compare results of the three algorithms: A-Priori in Task 1; PCY in Task 2; FPGrowth in Task 3 and report the accuracy of your implementation in Task 1 and Task 2. The objective is to evaluate and compare:

- The number of frequent itemsets that each algorithm finds.
- The number of association rules generated.
- The degree of overlap between the results of the algorithms and FP-Growth (considered as a benchmark).
- The percentage difference (%) between the algorithms compared to FP-Growth.

F. Initializing “FBGrowth” and Comparing the Methods

We conducted the following steps to achieve the results. First, transactional data stored on Google Drive was processed using “PySpark DataFrames” to ensure flexibility and high efficiency. The data was grouped by date (day, month, year) along with the purchased items.

Using the library “pyspark.ml.fpm import FPGrowth”, we have applied the “FBGrowth” algorithm to determine and calculate the number of frequent files and the number of association rules contained in the file “baskets.csv”. In order to compare the performance with other algorithms.

Transactions count: 14758

Member_number	Date	items
1000	15/03/2015	[whole milk, saus...
1000	24/06/2014	[pastry, whole mi...
1000	24/07/2015	[misc. beverages,...
1000	25/11/2015	[sausage, hygiene...
1000	27/05/2015	[pickled vegetabl...

only showing top 5 rows

Số lượng tập hợp phổ biến: FP-Growth=750

Số lượng luật kết hợp: FP-Growth=134

Fig. 12. Frequent pairs and association rules (FBGrowth)

After running the comparison program Apriori, PCY, and FP-Growth, based on the results we can draw the following main conclusions:

- FP-Growth often gives the fastest and most efficient results when processing large data. Because it does not need to generate and browse the entire candidate set like Apriori or PCY.
- Apriori is a classic algorithm, easy to understand but much slower when the number of items increases, because it has to scan the entire data many times.
- PCY is an improved version of Apriori, using hashing to reduce the number of candidates, so:
- The results are often similar to Apriori,
- But it saves more memory and time.

When compared with FP-Growth:

- The number of frequent itemsets and association rules of PCY and Apriori is usually a little smaller.
- The higher the overlap rate with FP-Growth → the more accurate the model.
- The percentage difference (%) helps quantify the difference between algorithms.

Số lượng tập phổ biến (FP-Growth): 750
Số lượng luật kết hợp (FP-Growth): 134
Tập phổ biến giao nhau với FP-Growth: Apriori=258, PCY=60

Frequent itemsets count (% difference from FP-Growth): Apriori=734.67%, PCY=-82.53
Association rules count (% difference from FP-Growth): Apriori=28023.88%, PCY=-2.2
Tỉ lệ giao nhau với FP-Growth (%): Apriori=34.40%, PCY=8.00%

=== Tổng kết ===
- Apriori: 6260 tập phổ biến, 37686 luật
- PCY: 131 tập phổ biến, 131 luật
- FP-Growth: 750 tập phổ biến, 134 luật

Conclusion:

- Apriori or PCY find more frequent itemsets than FP-Growth (possibly due to different parameters or counting methods).
 - Apriori or PCY generate more rules — however, they may contain redundant or duplicate rules.
 - Apriori has 34.40% frequent itemsets that match FP-Growth — higher than PCY (8.00%).
- Overall assessment:
- FP-Growth is faster and more efficient in processing large data because it does not need to generate many candidates.
 - Apriori is easier to understand but slower because it has to go through many loops.
 - PCY improves on Apriori by using hashing to reduce memory, but is still inferior to FP-Growth in large data.
- =>FP-Growth is the most efficient algorithm of the three algorithms in terms of speed and ability to mine large frequent itemsets.

IV. CONTRIBUTION

Contribution of each member:

Nguyen Triet Hung: 100%
Truong Nguyen Bao: 100%
Hoang Le Minh Nhat: 100%
Le Nguyen Hai An: 100%

V. SELF-EVALUTION

TASK 1: 100%
TASK 2: 100%
TASK 3: 100%
TASK 4: 100%

REFERENCES

- [1] Apache Hadoop, “MapReduce Tutorial,” Apache Software Foundation, [Online]. Available: <https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>. [Accessed: Apr. 6, 2025].
- [2] Baeldung, “MapReduce Algorithm,” [Online]. Available: <https://www.baeldung.com/cs/mapreduce-algorithm>. [Accessed: Apr. 6, 2025].
- [3] UniTrento, “PCY Algorithm,” [Online]. Available: <http://disi.unitn.it/themis/courses/MassiveDataAnalytics/slides/AssociationRules2-2in1.pdf>. [Accessed: Apr. 6, 2025].
- [4] Wikipedia, “FBGrowth” [Online]. Available: <https://en.wikipedia.org/wiki/FBGrowth>. [Accessed: Apr. 6, 2025].
- [5] IncludeHelp, “PCY Algorithm in Big Data Analytics,” [Online]. Available: <https://www.includehelp.com/big-data/pcy-algorithm-in-big-data-analytics.aspx>. [Accessed: Apr. 6, 2025].
- [6] Github, “FBGrowth Tutorial with Python Code,”

[Online]. Available: <https://github.com/adarsh-nl/FP-growth-algorithm-from-scratch/blob/main/fpgrowth.py> [Accessed: Apr. 6, 2025].

